

C Programming Assignment

Menu-Driven Digit Based Operations Using switch-case

Name	Gurveer Singh
Batch	15
SAF ID	590032925
Language	C Programming

Aim

To write a simple menu-driven C program using switch-case and separate void user-defined functions to check Strong, Armstrong, Harshad and Duck numbers.

The program is kept simple and readable so that each operation can be understood and explained easily.

1. Problem Statement

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer. Use one separate user-defined function for each menu option. Every function should have void return type, take no arguments, read its own input, process the number and display the result.

Option	Operation
1	Check whether the number is a Strong number.
2	Check whether the number is an Armstrong number.
3	Check whether the number is a Harshad number.
4	Check whether the number is a Duck number.

2. Basic Idea

Strong number: Sum of factorials of all digits is equal to the original number. Example: $145 = 1! + 4! + 5! = 145$.

Armstrong number: Sum of each digit raised to the number of digits is equal to the original number. Example: $153 = 1^3 + 5^3 + 3^3$.

Harshad number: The number is exactly divisible by the sum of its digits. Example: 18, because $1 + 8 = 9$ and 18 is divisible by 9.

Duck number: A positive number containing at least one zero in its digits. Example: 1023.

3. Algorithm

Main Program

1. Start the program.
2. Display four menu options.
3. Read the user choice.
4. Use switch-case to call the required function.
5. If the choice is invalid, display an error message.
6. Stop the program.

Strong Number

1. Read a positive integer n .
2. Store n in a temporary variable.
3. Take one digit at a time using $\% 10$.
4. Find the factorial of that digit and add it to sum.
5. Remove the last digit using $/ 10$.
6. If sum equals n , it is a Strong number; otherwise it is not.

Armstrong Number

1. Read a positive integer n .
2. Count the number of digits.
3. Again take every digit one by one.
4. Multiply the digit by itself as many times as the digit count and add the value to sum.
5. Compare the final sum with n and display the result.

Harshad Number

1. Read a positive integer n .
2. Find the sum of all its digits.
3. Check $n \% \text{sum}$.
4. If the remainder is 0, it is a Harshad number; otherwise it is not.

Duck Number

1. Read a positive integer n .
2. Check every digit one by one.
3. If any digit is 0, mark the number as a Duck number.
4. Display the result.

4. C Program

Simple implementation using four void functions with no parameters:

```
#include <stdio.h>

void strongNumber();
void armstrongNumber();
void harshadNumber();
void duckNumber();

int main() {
    int choice;

    printf("MENU\n");
    printf("1. Strong Number\n");
    printf("2. Armstrong Number\n");
    printf("3. Harshad Number\n");
    printf("4. Duck Number\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: strongNumber(); break;
        case 2: armstrongNumber(); break;
        case 3: harshadNumber(); break;
        case 4: duckNumber(); break;
        default: printf("Invalid choice\n");
    }
    return 0;
}

void strongNumber() {
    int n, temp, digit, i, fact, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;

    while (temp > 0) {
        digit = temp % 10;
        fact = 1;
        for (i = 1; i <= digit; i++)
            fact = fact * i;
        sum = sum + fact;
        temp = temp / 10;
    }

    if (sum == n)
        printf("%d is a Strong number.\n", n);
    else
        printf("%d is not a Strong number.\n", n);
}

void armstrongNumber() {
    int n, temp, digit, digits = 0, i, power, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;
    while (temp > 0) {
        digits++;
        temp = temp / 10;
    }

    temp = n;
    while (temp > 0) {
        digit = temp % 10;
        power = 1;
        for (i = 1; i <= digits; i++)
            power = power * digit;
        sum = sum + power;
        temp = temp / 10;
    }

    if (sum == n)
        printf("%d is an Armstrong number.\n", n);
    else
        printf("%d is not an Armstrong number.\n", n);
}
```

```

        temp = temp / 10;
    }

    if (sum == n)
        printf("%d is an Armstrong number.\n", n);
    else
        printf("%d is not an Armstrong number.\n", n);
}

void harshadNumber() {
    int n, temp, digit, sum = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;

    while (temp > 0) {
        digit = temp % 10;
        sum = sum + digit;
        temp = temp / 10;
    }

    if (n % sum == 0)
        printf("%d is a Harshad number.\n", n);
    else
        printf("%d is not a Harshad number.\n", n);
}

void duckNumber() {
    int n, temp, digit, found = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);
    temp = n;

    while (temp > 0) {
        digit = temp % 10;
        if (digit == 0) {
            found = 1;
            break;
        }
        temp = temp / 10;
    }

    if (found == 1)
        printf("%d is a Duck number.\n", n);
    else
        printf("%d is not a Duck number.\n", n);
}

```

5. Test Case Table

S.No.	Choice	Input	Expected Result
1	1	145	145 is a Strong number.
2	1	123	123 is not a Strong number.
3	2	153	153 is an Armstrong number.
4	2	125	125 is not an Armstrong number.
5	3	18	18 is a Harshad number.
6	3	19	19 is not a Harshad number.
7	4	1023	1023 is a Duck number.
8	4	1234	1234 is not a Duck number.

6. Sample Execution

MENU

1. Strong Number
2. Armstrong Number
3. Harshad Number
4. Duck Number

Enter your choice: 2

Enter a positive integer: 153

153 is an Armstrong number.

Result

The program was written using switch-case and four separate void functions. The prepared test cases give the expected results for Strong, Armstrong, Harshad and Duck number checks.